

Practica 2

1. ¿Cuál es la función de la capa de aplicación?

La función de la capa de aplicación es actuar como la capa más cercana al usuario y la que proporciona una interfaz y servicios de red directos a las aplicaciones de software

2. Si dos procesos deben comunicarse:

a. ¿Cómo podrían hacerlo si están en diferentes máquinas?

b. Y si están en la misma máquina, ¿qué alternativas existen?

Si dos procesos están en diferentes máquinas, deberán comunicarse mediante una red aplicando los protocolos estándar y así poder compartir información a distancia mediante un socket.

Si están en la misma máquina la comunicación no necesita de redes interconectadas, ya que la comunicación se realiza por medios locales (IPC).

3. Explique brevemente cómo es el modelo Cliente/Servidor. Dé un ejemplo de un sistema Cliente/Servidor en la “vida cotidiana” y un ejemplo de un sistema informático que siga el modelo Cliente/Servidor. ¿Conoce algún otro modelo de comunicación?

En el modelo Cliente/Servidor existe un cliente el cual le realiza peticiones a un servidor, el cual escucha pasivamente esperando esas peticiones, procesándolas y devolviendo lo pedido

Un ejemplo de la vida cotidiana es un cajero automático, el cual a la hora de necesitar el saldo de la persona, le consulta al banco sobre el mismo y este le responde

Otro modelo es el P2P (Peer to Peer) en el cual no hay jerarquías sino que ambos pueden actuar tanto como clientes o como servidores

4. Describa la funcionalidad de la entidad genérica “Agente de usuario” o “User agent”.

Un agente de usuario es la aplicación que actúa en nombre de un usuario en una red. Es un componente que representa al usuario en sus interacciones con sitios web, servicios en

linea, etc.

En el contexto web se refiere al navegador que un usuario usa. En una solicitud HTTP el User Agent es un encabezado que informa sobre el navegador y SO

5. ¿Qué son y en qué se diferencian HTML y HTTP?

HTTP Se refiere a un estanda para realizar peticiones a un servidor. Tiene un formato definido, headers, tipo, contenido, etc.

HTML es un tipo de notacion de texto, sobre el cual estan hechas las paginas web. Se basa en etiquetas y contenido

6. HTTP tiene definido un formato de mensaje para los requerimientos y las respuestas.

a. ¿Qué información de la capa de aplicación nos indica si un mensaje es de requerimiento o de respuesta para HTTP? ¿Cómo está compuesta dicha información?¿Para qué sirven las cabeceras?

b. ¿Cuál es su formato?

c. Suponga que desea enviar un requerimiento con la versión de HTTP 1.1 desde curl/7.74.0 a un sitio de ejemplo como www.misitio.com para obtener el recurso /index.html. En base a lo indicado, ¿qué información debería enviarse mediante encabezados? Indique cómo quedaría el requerimiento.

a)

Formato de mensaje de una solicitud

```
1 GET /unadireccion/pagina.html HTTP/1.1
2 Host: www.unaEscuela.edu
3 Connection: close
4 User-agent: Mozilla/5.0
5 Accept-language: fr
```

Primera linea: linea de requerimiento, tiene 3 campos

- Campo de metodo (GET, PUT, POST)

- Campo de URL
 - Campo de version HTTP
- Siguientes lineas: Headers

Mensaje de respuesta

```
1  HTTP/1.1 200 OK
2  Connection: close
3  Date: Tue, 18 Aug 2015 15:44:04 GMT
4  Server: Apache/2.2.3 (CentOS)
5  Last-Modified: Tue, 18 Aug 2015 15:11:03 GMT
6  Content-Length: 6821
7  Content-Type: text/html
8
9  (datos datos datos datos datos ... )
```

Tres secciones: estado inicial, seis headers y un cuerpo

La línea de estado especifica la version del protocolo, un código de estado y el mensaje explicativo del estado

El cuerpo de identidad representa el objeto solicitado en si

Header: Son campos de metainformacion en formato clave: valor, sirven para negociar, describir y controlar como se lleva a cabo la comunicacion y como procesar los datos del body

b)

El formato de las cabeceras/headers es → <Nombre de la cabecera>:<Valor de la Cabecera> Cada cabecera ocupa una línea y el bloque de cabeceras termina con una línea en blanco que indica el final de las cabeceras y el comienzo del cuerpo del mensaje (si lo hay).

c)

```
1 GET /index.html HTTP/1.1
2 Host: www.misitio.com
3 User-Agent: curl/7.74.0
4 Connection: close
```

7. Utilizando la VM, abra una terminal e investigue sobre el comando curl. Analice para qué sirven los siguientes parámetros (-I, -H, -X, -s).

El curl sirve para hacer una petición HTTP a una página, por ejemplo `curl google.com`, el método por defecto es un `GET`

Parámetros:

- `-I` (`--head`) : Manda una petición `HEAD` en vez de un `GET`
 - `-H` (`--header`) : Deja agregarle un header a la petición
 - `-X` (`--request`) : Fuerza el método HTTP, se le pasa el método como `curl -X DELETE https://httpbin.org/delete`
 - `-s` (`--silent`) : Oculta la barra de progreso y los mensajes de error, solo muestra la salida
-

8. Ejecute el comando curl sin ningún parámetro adicional y acceda a www.redes.unlp.edu.ar. Luego responda:

- a. ¿Cuántos requerimientos realizó y qué recibió? Pruebe redirigiendo la salida (`>`) del comando curl a un archivo con extensión `html` y abrirlo con un navegador.
 - b. ¿Cómo funcionan los atributos `href` de los tags `link` e `img` en `html`?
 - c. Para visualizar la página completa con imágenes como en un navegador, ¿alcanza con realizar un único requerimiento?
 - d. ¿Cuántos requerimientos serían necesarios para obtener una página que tiene dos `CSS`, dos `Javascript` y tres imágenes? Diferencie cómo funcionaría un navegador respecto al comando curl ejecutado previamente.
- a) Se pidió una sola cosa y se recibió una respuesta HTTP (`GET /`), sin ningún `CSS` ni los recursos que se referencian
 - b) No contienen al recurso como tal, sino un link al recurso mismo. Al traer el `HTML` eso se

pierde

- c) Una petición HTTP trae un recurso por vez, para traerlo completo habría que hacer una petición a cada referencia y estilo de la página
 - d) Serían 8 peticiones, una para el esqueleto HTML y otra para sus referencias. La diferencia entre el curl y el navegador es que el curl necesita hacer esas peticiones por separado, cuando el navegador hace las 8 a la vez
-

9. Ejecute a continuación los siguientes comandos:

- `curl -v -s www.redes.unlp.edu.ar > /dev/null`
- `curl -I -v -s www.redes.unlp.edu.ar`
 - a. ¿Qué diferencias nota entre cada uno?
 - b. ¿Qué ocurre si en el primer comando se quita la redirección a `/dev/null`? ¿Por qué no es necesaria en el segundo comando?
 - c. ¿Cuántas cabeceras viajaron en el requerimiento? ¿Y en la respuesta?

La principal diferencia es que una es un método GET y la otra un método HEAD. Este último devuelve status + cabeceras, mientras que el segundo eso sumando el cuerpo de la petición

Ambos no tienen el cuerpo de la petición, uno nunca lo pide porque es un HEAD y el otro manda el body al `/dev/null`. Uno directamente no pide el body y el otro en la respuesta

En el requerimiento viajaron 3, en la respuesta viajaron 7

10. ¿Qué indica la cabecera Date?

La cabecera Date indica la fecha y hora en la que el servidor generó la respuesta

Date: Tue, 08 Sep 2026 14:32:07 GMT

11. En HTTP/1.0, ¿cómo sabe el cliente que ya recibió todo el objeto solicitado de manera completa? ¿Y en HTTP/1.1?

- En HTTP/1.0 se sabe que finalizo la recepcion de un recurso ya que el servidor cierra la conexion despues de enviar toda la informacion. Esto se debe a que usa conexiones no persistentes
 - En HTTP/1.1 se sabe que finalizo la recepcion con el header `Content-length`, con `Chunked Transfer Encoding` si no se sabe el tamaño de antemano, se sabe que finalizo cuando llega un fragmento de tamaño 0 o con el cierre explicito de la conexion con el header `Connection: close`
-

12. Investigue los distintos tipos de códigos de retorno de un servidor web y su significado. Considere que los mismos se clasifican en categorías (2XX, 3XX, 4XX, 5XX).

En general los estados 100 son informativos, los 200 indican que la peticion tuvo exito, los 300 indican una redireccion, los 400 un error por parte de la peticion/cliente y los 500 un error del lado del servidor

13. Utilizando curl, realice un requerimiento con el método HEAD al sitio www.redes.unlp.edu.ar e indique:

- ¿Qué información brinda la primera línea de la respuesta?
 - ¿Cuántos encabezados muestra la respuesta?
 - ¿Qué servidor web está sirviendo la página?
 - ¿El acceso a la página solicitada fue exitoso o no?
 - ¿Cuándo fue la última vez que se modificó la página?
- f. Solicite la página nuevamente con curl usando GET, pero esta vez indique que quiere obtenerla sólo si la misma fue modificada en una fecha posterior a la que efectivamente fue modificada. ¿Cómo lo hace? ¿Qué resultado obtuvo? ¿Puede explicar para qué sirve?
- Dice la version del HTTP, el codigo de respuesta y una descripcion del estado
 - Muestran 7 headers
 - Apache/2.4.56 (Unix), te lo dice el encabezado `Server:`
 - Fue exitoso, la primera linea nos informa un codigo `200 OK`
 - El domingo 19 de marzo del 2023, lo dice el encabezado `Last-Modified:`
 - `curl -H "If-Modified-Since: Sun, 19 Mar 2023 19:04:46 GMT" www.redes.unlp.edu.ar`
-

14. Utilizando curl, acceda al sitio www.redes.unlp.edu.ar/restringido/index.php y siga las instrucciones y las pistas que vaya recibiendo hasta obtener la respuesta final. Será de utilidad para resolver este ejercicio poder analizar tanto el contenido de cada página como los encabezados.

```
redes@debian:~$ curl -H "Authorization: Basic cmVkZXN6U1lD" www.redes.unlp.edu.ar/restringido/the-end.php
¡Felicitaciones, llegaste al final del ejercicio!

Fecha: 2026-09-17 15:26:17
Verificación: cbf03292972ca67c04be3dbc99059d0738f2b8ab27075cf6be87dc11ccc042abredes@debian:~$
```

15. Utilizando la VM, realice las siguientes pruebas:

a. Ejecute el comando 'curl www.redes.unlp.edu.ar/extras/prueba-http-1-0.txt' y copie la salida completa (incluyendo los dos saltos de línea del final).

b. Desde la consola ejecute el comando telnet www.redes.unlp.edu.ar 80 y luego pegue el contenido que tiene almacenado en el portapapeles. ¿Qué ocurre luego de hacerlo?

c. Repita el proceso anterior, pero copiando la salida del recurso [/extras/prueba-http-1-1.txt](http://www.redes.unlp.edu.ar/extras/prueba-http-1-1.txt). Verifique que debería poder pegar varias veces el mismo contenido sin tener que ejecutar el comando telnet nuevamente.

a.

```
1 GET /http/HTTP-1.1/ HTTP/1.0
2 User-Agent: curl/7.38.0
3 Host: www.redes.unlp.edu.ar
4 Accept: */*
5
6
```

b.

Lo que ocurre es que se devuelven las cabeceras y contenidos, y luego aparece
Connection closed by foreign host

c.

Lo que pasa aca se lo mismo pero al devolver todo la conexion queda abierta

16. En base a lo obtenido en el ejercicio anterior, responda:

- a. ¿Qué está haciendo al ejecutar el comando telnet?**
 - b. ¿Qué método HTTP utilizó? ¿Qué recurso solicitó?**
 - c. ¿Qué diferencias notó entre los dos casos? ¿Puede explicar por qué?**
 - d. ¿Cuál de los dos casos le parece más eficiente? Piense en el ejercicio donde analizó la cantidad de requerimientos necesarios para obtener una página con estilos, javascripts e imágenes. El caso elegido, ¿puede traer asociado algún problema?**
- a. Con telnet estamos abriendo un canal de comunicacion mediante el protocolo TCP para enviarnos recursos con el servidor web
 - b. Se hizo un requerimiento GET, solicitando el html de la pagina
 - c. En el primer caso la conexon se cerro despues de recibir el recurso, en el otro caso la conexon permanecio abierta
 - d. Parece mas eficiente HTTP/1.1 ya que no se abre una conexon por cada recurso, para miles de recursos esto se torna ineficiente. Esto puede traer consumo excesivo de sockets
-

17. En el siguiente ejercicio veremos la diferencia entre los métodos POST y GET. Para ello, será necesario utilizar la VM y la herramienta Wireshark.

- a. Abra un navegador e ingrese a la URL: www.redes.unlp.edu.ar e ingrese al link en la sección “Capa de Aplicación” llamado “Métodos HTTP”. En la página mostrada se visualizan dos nuevos links llamados: Método GET y Método POST. Ambos muestran un formulario.**
 - b. Analice el código HTML**
 - c. Utilizando el analizador de paquetes Wireshark capture los paquetes enviados y recibidos al presionar el botón Enviar.**
 - d. ¿Qué diferencias detectó en los mensajes enviados por el cliente?**
 - e. ¿Observó alguna diferencia en el browser si se utiliza un mensaje u otro?**
- b. El codigo HTML de las consultas muestra que en el form tiene como atributo `method="GET"` y `action="metodos-lectura-valores.php"`
 - d. Los datos en el GET van en la URL,
`https://www.redes.unlp.edu.ar/registro.php?nombre=Juan&apellido=Perez&edad=25`
En el post los datos van en el cuerpo del mensaje, se agregan algunos encabezados
 - e. Con el GET, los datos quedan en la URL, si recargo la pagina no pasa nada. Con POST, la barra de direcciones no expone datos, cuando quiero recargar me tura una advertencia
-

18. Investigue cuál es el principal uso que se le da a las cabeceras Set-Cookie y Cookie en HTTP y qué relación tienen con el funcionamiento del protocolo HTTP.

Relacion con el protocolo HTTP: HTTP no tiene estado, las cookies permiten a las aplicaciones web mantener un estado

Set-Cookie: El servidor me pasa un cookie para que yo, como cliente, la pase en cada petición

Cookie: Le paso una cookie que me seteo el servidor antes

19. ¿Cuál es la diferencia entre un protocolo binario y uno basado en texto? ¿De qué tipo de protocolo se trata HTTP/1.0, HTTP/1.1 y HTTP/2?

El protocolo de texto usa caracteres legibles por humanos con saltos de línea. El binario divide los metadatos, métodos y mensajes en *frames*. La ventaja del texto plano es que es legible para humanos pero genera mayor sobrecarga de datos, en cambio el binario es más eficiente para navegadores pero no se puede leer sin procesarlo

Version	Calificación
HTTP/1.0	Texto plano
HTTP/1.1	Texto plano
HTTP/2	Binario

20. Responder las siguientes preguntas:

a. ¿Qué función cumple la cabecera Host en HTTP 1.1? ¿Existía en HTTP 1.0? ¿Qué sucede en HTTP/2?

b. En HTTP/1.1, ¿es correcto el siguiente requerimiento?

1 GET /index.php HTTP/1.1

2 User-Agent: curl/7.54.0

c. ¿Cómo quedaría en HTTP/2 el siguiente pedido realizado en HTTP/1.1 si se está usando https?

- a. En HTTP/1.1, Host indica el nombre del dominio al que se quiere acceder, esto permite hosts virtuales para que una misma ip tenga varios dominios. En HTTP/1.0 no existia al principio y despues se agrego opcional. En HTTP/2 se reemplaza por el pseudo-encabezado authority
- b. No es correcto ya que le falta la cabecera obligatoria Host, sin esto el servidor va a tirar 400 BAD REQUEST y porque falta la linea CRLF al final de los encabezados
- c. En HTTP/2, los métodos, rutas y metadatos no viajan en texto plano como la primera línea tradicional, sino codificados como **pseudo-encabezados** (que comienzan con :) dentro de un marco binario de cabeceras (*HEADERS frame*)

```
1 :method: GET
2 :path: /index.php
3 :scheme: https
4 :authority: www.redes.unlp.edu.ar
5 user-agent: curl/7.54.0
```
